

Reliable Bytes Can Still Be Dangerously Old

Project Hands-on Day 2: Keep It Fresh
`frame_v1` $\longrightarrow$ `transport_v1`

Mentor – Prof. Dr. Mohammed Aquil Mirza, COMP
Student Mentors – Mr. Jiang Suyu, Mr. Ruan Haozhe

July 2026
Summer Course at Lanzhou University
The Hong Kong Polytechnic University (PolyU)

frame_v1 is today's only transport payload

Run once before editing

```
make verify-day1
```

If the gate is red, record the failed test and switch to the latest class-generated frame_v1 checkpoint. Do not repair yesterday's frame contract during this hour.

Begin transport work only when

- version, length, and CRC tests are green;
- seq and t_source_ns survive round trip;
- corrupt input still produces zero actuator writes.

```
Frozen:  FrameV1 / CRC / sequence / source time / units
```

A delayed valid command can overwrite a newer command

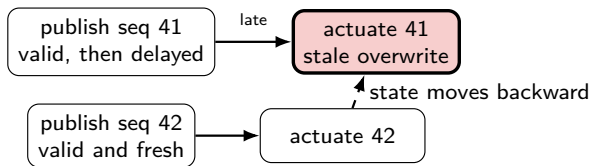
Reproduce the stale-overwrite path

```
make demo-day2-stale
```

The fixture delays valid sequence 41, delivers sequence 42, then releases 41. Both frames pass the integrity gate.

Capture this signature

actuated_seq: 42 $\longrightarrow$ 41



CRC proves integrity; QoS proves a delivery attempt. Neither proves that the command is still useful.

Broken contract: freshness, not MQTT reliability.

QoS, sequence, age, and latest value answer different questions

Signal	Frozen delivery rule
arm/state	QoS 0; keep latest
arm/cmd	QoS 1; reject non-new seq
arm/estop	QoS 1; preserve source age

QoS: should MQTT retry?

Sequence: is this newer?

Age: is it still useful?

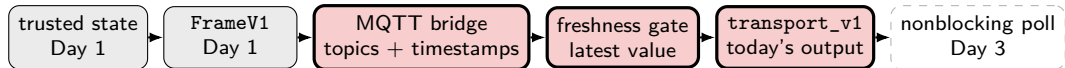
Latest value: what should a late reader see?

Soda Code

```
fn accept(g: &mut FreshnessGate, f: &FrameV1,
          now_ns: u64) -> Result<(), Reason> {
    require(frame_valid(f), Reason::BadFrame)?;
    require(f.seq > g.last_seq, Reason::NotNew)?;
    require(now_ns >= f.t_source_ns,
            Reason::ClockError)?;
    require(now_ns - f.t_source_ns <= MAX_CMD_AGE_NS,
            Reason::Stale)?;
    g.last_seq = f.seq;
    Ok(())
}
```

Compare timestamps only after converting them to the frozen experiment timebase. Update the latest-value store only after accept returns Ok.

Day 2 inserts MQTT without weakening the frame gate



Writable today

`mqtt_bridge`, `topics`, `freshness`, `transport`
fixtures, clocks, and raw logs.

Read-only today

Frame layout, CRC, state schema, units, sequence
meaning, and simulator actuator interface.

Input `frame_v1` → **Output** `transport_v1`

Fifteen students place freshness across all ten chapters

Team	Build	Deliver	Pass
G1 1/2	Publish on the frozen clock; inject duplicate and stale state.	Pose-age data + clock note	Stale is flagged; fresh stays latest.
G2 3/4	Set topics/QoS; implement sequence, age, and latest value.	transport_v1 + topic table	Invalid, duplicate, or stale is not forwarded.
G3 5/6	Create congestion; locate broker, kernel, and user queues.	Queue trace + queue map	The delayed layer is named from evidence.
G4 7/8	Map age, QoS, clock, and disconnect to safety inputs.	Freshness fault table	Every stale reject has a stable reason.
G5 9/10	Trace controller → frame → MQTT → simulator.	End-to-end trace	Input, verdict, and output share one trace ID.

Each team: 3 students / 2 chapters / 1 reproducible evidence bundle

Publish, delay, measure, and keep only accepted work

Soda Code

```
mqtt.subscribe(CMD_TOPIC, Qos::AtLeastOnce)?;

fn on_message(b: &mut Bridge, bytes: &[u8],
              now_ns: u64) -> Result<(), Reason> {
    let f = decode(bytes)?;
    // TODO 1: stamp t_rx, then run freshness.accept.
    // TODO 2: replace latest only after acceptance.
    // TODO 3: log four stamps + verdict + reason.
    Ok(())
}
```

Run three workloads

normal publish
QoS 1 duplicate
delayed old frame

Three-person split

1 – topics + publisher
2 – freshness + latest value
3 – clocks + raw evidence

Minute 11

green, blocked, or
using class checkpoint

Frozen during the sprint: FrameV1, topic names, QoS choices, timestamp meanings, workload seed, and age threshold.

Freshness passes only when every verdict is traceable

Run the canonical gate

```
make verify-day2
```

Save inside Gxx_D2_evidence/:

- qos-freshness.csv
- topic-table.txt
- trace.jsonl
- manifest.json

All three claims must be reproducible

1. invalid, duplicate, or stale input reaches the actuator **zero** times;
2. every row records seq, t_pub, t_rx, t_gate, t_ack, verdict, and reason;
3. normal, duplicate, and delayed workloads replay from the recorded command, seed, commit, and clock-offset parameters.

Finish both chapter evidence files: 2–3 method bullets plus one figure or table caption.

Day 3 will keep fresh messages moving past slow clients

Input `frame_v1` → **Output** `transport_v1`

Freeze and hand off

- topic table and QoS choices;
- timebase and four timestamp meanings;
- sequence, latest-value, and age policy.

Before tagging

- name the artifact owner;
- record one known issue or none;
- verify G3/G5 replay the evidence bundle.

Tag only after green: `git tag -a transport_v1 -m "Day 2 gate passed"`

Day 3 preflight: `make verify-day2`

Thanks for listening

Presented by Suyu Jiang

Template: Azusa Template